

E-Closet

Recommendation System

Implementation Plan & Technical Review

1. Project Overview

Goal: Replace the current generic category feed with a personalized feed that shows each user items they are most likely to view, save, add to cart, and rent or purchase.

1.1 Approach

A content-based + behavioral hybrid scoring system built on top of pre-trained multimodal embeddings. Deliberately avoids training any neural network from scratch at this stage because:

- Labeled data is small and sparse (~9.5k items, ~7.5k users, ~25k interactions)
- Cold-start is the dominant use case
- Explainable, debuggable scores are preferred over black-box models

1.2 Decisions Made

Decision	Outcome	Status
Base model	FashionSigLIP (Marqo/marqo-fashionSigLIP)	Confirmed
Fine-tuning	Not yet — need 3-4 weeks of real data first	Deferred
Embedding storage	MongoDB (clothes_embeddings, user_embeddings)	Confirmed
Data sources	MongoDB (catalog, profile, orders) + BigQuery (events)	Confirmed
Refresh cadence	Nightly Cloud Run Job via Cloud Scheduler (3am)	Confirmed
Remove-from-cart penalty	Scale to 40% of positive signal (not zero out)	Updated
Launch target for behavior tracking	Friday (tomorrow) deploy backend + iOS together	In progress

2. Algorithm Design

2.1 Item Embeddings

Each item gets one 768-dimensional vector, computed offline nightly:

- Render text: "{brand} {title}. {categories}. {description}. {color}."
- Encode text with FashionSigLIP (768-d output)
- Download primary image, encode with same model (image tower)
- Fuse: $\text{item_emb} = 0.5 * \text{text_emb} + 0.5 * \text{image_emb}$, then L2-normalize
- Items with missing images fall back to text-only

2.2 User Embeddings

Every user vector is a weighted blend of two components:

Behavioral Component

Weighted average of item vectors the user has interacted with, using event strength x time decay (30-day half-life).

Event	Weight
Purchase (completed order)	10.0
Purchase (MongoDB order metadata, delivered)	8.0
Begin checkout	6.0
Rental completed	10.0
Add to cart	5.0
Message seller	4
Favorite / like	4
Opened delivery-date picker	3
Bookmark	3.5
Product detail viewed	2
Product card / image click	1
Search submitted	2
Remove from cart	Scale remaining signal to 40%
Scroll-past (slow)	-0.05
Scroll-past (fast)	-0.10
Scroll-past 3+ times (suppression)	Suppress for 30 days

Remove-from-cart no longer zeros out all positive signals. Instead, the remaining net positive signal is scaled to 40%. This preserves the real intent signal of adding to cart while discounting for the removal.

Scroll-past suppression is triggered by repeated scroll-past events only (3+ across sessions with no click). Users cannot manually remove items from their feed, so remove-from-cart is not part of the suppression trigger.

Declared Component

SigLIP text embedding of the user's style and occasion tags from onboarding (e.g. "Style: streetwear, classy. Occasion: gameday, formal.").

Fusion Formula

$\alpha = 1 / (1 + \text{total_behavior_weight})$

$\text{user_emb} = \text{normalize}(\alpha * \text{declared_emb} + (1 - \alpha) * \text{behavioral_emb})$

- No behavior, has tags -> declared dominates (cold start)
- Heavy behavior -> declared tags become effectively irrelevant
- Neither -> falls back to popularity + same-city non-personalized path

2.3 Item Suppression & Similarity Boost

Once a user has a strong positive interaction with an item, showing that exact item again is wasted feed real estate. Instead, the interacted item becomes a taste signal to surface similar items the user has not yet seen.

Suppression Rules: hide the exact item for a time window

Each interaction tier triggers a different suppression duration. After the window expires, the item re-enters the candidate pool naturally.

Interaction	Suppression Window	Rationale
Purchase / rental completed	90 days	Just bought or rented — no need to see it again soon. Fashion is cyclical so re-surface after ~3 months.
Begin checkout (did not complete)	30 days	Strong intent but something stopped them. Give it space, then re-surface in case they want to revisit.
Add to cart (not checked out)	14 days	High interest signal. Suppress briefly so similar items get exposure, but re-surface soon in case they still want it.

Interaction	Suppression Window	Rationale
Favorite / bookmark	7 days	They saved it deliberately — they know it exists. Show similar items now, re-surface the original after a week.
Product detail viewed	No suppression	Viewing alone is not strong enough to suppress. MMR handles over-repetition.
Scroll-past 3+ times	30 days	Clear disinterest. Suppress and re-evaluate after a month in case taste changes.

Suppression windows are stored as `suppressed_until` timestamps on the user-item pair in MongoDB. The hard filter in the candidate pool builder checks this field before scoring. No item with `suppressed_until > today` is ever scored.

Similarity Boost: surface items like what they loved

When an item is suppressed due to a strong positive interaction (add to cart, purchase, rental), its embedding is used to boost similar unseen items in the same scoring pass. This is what turns a single strong signal into a richer feed of related items.

- For each item in the user's high-signal suppressed set, compute cosine similarity against all candidate items
- Take the max similarity score across all suppressed items as the `similarity_boost` signal
- Normalize to `[0, 1]` and add as a 6th score channel at weight 0.05
- Only items the user has NOT already interacted with are eligible — the boost surfaces new discoveries, not repeats

Trigger	Added to similarity boost pool?
Purchase / rental completed	Yes — strongest taste signal
Begin checkout	Yes
Add to cart	Yes
Favorite / bookmark	Yes
Product detail viewed	No — too weak, would dilute the signal
Scroll-past suppression	No — disinterest signal, not a taste signal

Example: a user adds a red floral midi dress to cart. That dress gets suppressed for 14 days. Its embedding is used to boost other red floral midi dresses and similar styles in the feed. The user sees fresh options they have not seen before, not the same item on repeat.

2.4 Scoring Formula

score =

- 0.38 . cosine(user_emb, item_emb)
- 0.17 . same_city_or_state_boost [1.0 same city, 0.5 same state]
- 0.14 . popularity_prior [log(1 + likes) * avg_rating]
- 0.09 . recency_prior [decay on listing age, 60-day half-life]
- 0.12 . neighbor_signal [collaborative filtering — see Section 3.2]
- 0.10 . similarity_boost [max cosine sim to high-signal suppressed items]

Weights sum to 1.0. These are hand-picked starting values. Learning-to-rank over these weights is a Phase 2 goal once enough labeled data is collected.

2.5 Hard Filters (applied before scoring)

Applied to the candidate pool before any scoring occurs. Items failing any filter are never scored.

- Item status not in {sold, unavailable, deleted, archived}
- Item seller is not the requesting user
- Item has no active suppression window for this user (suppressed_until <= today)
- Item's parent category matches user's inferred gender (if inferable)
- Item routing_mode matches user's eligibility (local vs shipping)

The interaction history filter from the previous version has been replaced by the suppression window system. Items are no longer excluded forever just because a user viewed them — only strong positive interactions and repeated scroll-pasts trigger time-bounded suppression.

3. Additional Scoring Signals

3.1 Collaborative Filtering (Neighbor Signal)

Uses existing user embeddings to find similar users in the same city, then aggregates their item interactions as an additional score channel.

- Find top-20 nearest users in the same city by cosine similarity of user embeddings
- Aggregate their weighted interactions, scaled by similarity score
- Normalize to [0,1] and add as the neighbor_signal score channel (weight: 0.12)
- Use a 14-day time window to surface what is trending locally right now
- Minimum neighbor threshold: if fewer than 5 same-city users have behavioral data, fall back to state-level, then skip signal entirely

This gives the algorithm a TikTok-like quality, it naturally surfaces seasonal and trending items without any hardcoding. What similar users in the same city are engaging with this week automatically flows into each user's feed.

3.2 Rental Urgency Signal

Adjusts scoring based on how soon the user needs the item, derived from the delivery-date picker event (already tracked at weight 2.5).

- 0-3 days out: local items only (urgency boost = 1.0 local, 0.0 non-local)
- 4-14 days out: partial local preference (0.6 local, 0.3 non-local)
- 15+ days out: distance matters less, style match matters more (0.2 flat)

4. V1 Implementation Plan

Three phases must be completed in sequence. The algorithm cannot serve real users until all three are live.

Phase 1: Backend Job (E-Closet-RecSys)

Task	Repo	Priority
Dockerfile with FashionSigLIP weights baked in (~1.2 GB, CPU-only torch)	RecSys	Launch blocker
Replace events.csv reader with live BigQuery client query	RecSys	Launch blocker
Cloud Run Job manifest + Cloud Scheduler cron (0 3 * * *)	Infra	Launch blocker
Mongo upsert writer for clothes_embeddings and user_embeddings	RecSys	Launch blocker
Add health_status, routing_mode, local_impressions_30d fields to schema	RecSys	Before launch
Add impression counter (total_impressions, unique_impressions) to nightly upsert	RecSys	Before launch

Phase 2: API Endpoint (E-Closet-Apis)

Task	Repo	Priority
New route: GET /recommendations?user_id=...&k=...	Apis	Launch blocker
Scoring controller: load embeddings, apply formula, return top-K	Apis	Launch blocker
Mongo indexes on cloth_id and user_id on embedding collections	DB / Infra	Launch blocker
Candidate pool filter with routing_mode logic	Apis	Week 1 post-launch

Phase 3: iOS Integration (E-Closet-iOS)

Task	Repo	Priority
Update home feed view model to call /recommendations	iOS	Launch blocker
Keep category feed as fallback if endpoint is unavailable	iOS	Launch blocker

Schema Fix — Any Phase

Add `created_time` and `updated_time` to the favorite and bookmark Mongo schemas. Currently no timestamp exists, so time-decay on those signals is approximated from ObjectId generation time. Collect accurate data from day one.

5. Diversity & Feed Guardrails

All diversity techniques are post-scoring layers. The algorithm scores freely, diversity is enforced separately so it never needs to be retrained to tune.

5.1 Maximal Marginal Relevance (MMR)

Penalizes items too similar to ones already selected for the feed. A lambda parameter (start at 0.7) controls relevance vs diversity tradeoff. Uses existing item embeddings, no new data needed. Drop-in replacement for the current top-K slice in `retrieve.py`.

Ship this week. Runtime is $O(k^2)$ at 20 items — negligible cost.

5.2 Slot Rules

Hard caps applied after scoring to ensure feed variety:

- Maximum 4 items per category in any top-20 feed
- Maximum 2 items from the same seller in any top-20 feed
- Maximum 2 stale/rerouted items per feed (seller-side guardrail)

Ship this week. 10 lines in `retrieve.py`. The seller cap directly prevents any single seller's listings from dominating, regardless of how many items they have.

5.3 Exploration Slots

Reserve 3 of 20 slots per feed for items with fewer than 5 total impressions across the platform. Inserts at positions 8, 14, 19 — not at the top. Surfaces new listings and supports sellers whose items have not gotten traction yet.

Implement week 2 post-launch. Needs real impression data to know what is underexposed.

5.4 Global Impression Cap

Once an item has been seen by more than 30% of the active user base, apply a 0.4x soft penalty to its `popularity_prior` score. Breaks the feedback loop where viral items keep compounding forever.

Phase 2. Only meaningful once some items have actually saturated the user base — roughly 4-6 weeks post-launch.

6. Seller-Side Guardrail

Items that are not getting local traction are automatically rerouted to shipping-eligible users nationally. This is a pre-scoring eligibility layer, the recommendation algorithm itself never changes.

6.1 Item Health Classification

Runs nightly in the Cloud Run Job alongside embedding computation. Assigns one of three states to every item:

Status	Criteria	Routing
Healthy	10+ local views OR 2+ saves in past 30 days	Local only
Struggling	Low local engagement, within grace period (14 days)	Local + shipping
Stale	30+ days, fewer than 5 local views	Shipping only

Recovery: if a stale item gets 5+ local views after being rerouted, it graduates back to 'both' mode.

Seller transparency: Phase 2 goal is a seller dashboard showing health_status and routing_mode so sellers know when their items are being rerouted and why. Do not do this silently forever.

6.2 Tunable Thresholds

Parameter	Starting Value	Notes
Stale threshold	30 days	Adjust based on average rental cycle length
Local views for healthy	10 views	Adjust once production view counts are known
Struggling grace period	14 days	How long before escalating to stale
Max stale items per feed	2 of 20	Protects buyer feed quality
Recovery threshold	5 local views	To graduate back from stale to both

7. Evaluation & Performance

7.1 Evaluation Metrics Scorecard

Two separate metric layers, offline (measurable now) and online (post-launch). Do not compare them directly.

Metric	Target	When	Answers
Category match rate	90%+	Offline now	Is the feed directionally correct?
Catalog coverage	60%+	Offline now	Are sellers getting exposure?
Hit@10 / NDCG	Track delta	Offline always	Is the model improving? (internal only)
Session engagement rate	85% aspirational	Post-launch	Is the feed driving real clicks?
Add-to-cart rate from recs	Baseline TBD	Post-launch	Is the feed driving rentals?

The 85% target is officially attached to category match rate for now, measurable today. After 3-4 weeks of production data, migrate the primary target to session engagement rate, which is a stronger signal of real business value.

9. Open Items & Known Gaps

Item	Notes	Priority
favorites/bookmarks missing timestamps	Derive from ObjectId until schema is updated	Before launch
begin_checkout / add_to_cart events not in BQ export	Confirm if missing from sample or not yet firing	Verify now
dwel_ms missing from product_detail_viewed	Phase 2 boost weight for long dwell times	Phase 2
user.accepts_shipping field does not exist	Default to true for now; add preference later	Phase 2
Dev vs prod data separation	Prevent dev interactions from polluting user embeddings	Before launch
Cloud Run Job timeout	Benchmark embedding rebuild time; likely needs 4-6hr timeout	Before launch
Deduplication of near-identical items	Same image, different listings, suppress duplicates at scoring time	Phase 2